

关于 DG-202606 智慧零售赛题仿真镜像变更的说明

发布单位：锐捷网络股份有限公司
适用题目：DG-202606 面向智慧零售的自主服务机器人研发与应用
适用阶段：初赛正式评测
文件版本：V2.0（2026-07-31）
对照文件：原《面向智慧零售的自主服务机器人研发与应用比赛方案》
配套文件：README.md、server物品清单.md

各参赛团队：

为使参赛团队能够在统一的仿真场景、机器人模型和 ROS2 环境中开展开发调试，发榜单位现提供新版智慧零售 Server 镜像、Client 镜像及固定 Baseline 示例。现将新版镜像与原比赛方案相比发生变化的内容说明如下。

一、本次说明的适用范围

本说明仅对新版 Docker 镜像所对应的比赛场景、任务下发方式、商品布局、障碍布局、机器人仿真平台、ROS2 接口和示例程序作补充和变更说明。

初赛赛制、任务数量和评分以正式比赛方案及发榜单位最新通知为准；镜像部署、启动命令、运行参数和 ROS2 接口以开发团队编写的 README.md 及对应 Server/Client 镜像为准。

原比赛方案中以下内容不变：

1. 参赛对象及组队要求；
2. 比赛时长为 10 分钟/队；
3. 每局下发 5 个订单；
4. 必须按照“取一件 -> 送一件 -> 再取下一件”的模式执行；
5. 任务完成分 80 分、技术评价分 20 分；
6. 作品提交内容、提交时间和报名方式；
7. 奖项、奖励措施、赛事保障及联系方式。

新版镜像完整场景中包含45件商品。README 中的 SUPERMARKET_TASKS=all 用于参赛团队进行全场功能开发和压力测试；正式比赛仍按5个订单下发，具体目标、随机种子和启动参数以发榜单位赛时配置为准。

二、主要变化对照

项目	原比赛方案	新版镜像
场景货架	3~5组标准货架	固定为5组货架
货位数量	未明确总数	每组3层、每层3列，共45个货位

项目	原比赛方案	新版镜像
商品种类	按五大类物品清单准备	镜像内设置9个 kind
商品数量	未明确场景总数	每个 kind 5件，共45件
商品位置	订单下发精确 location_id	每局随机分配，不下发位置
ArUco	每件商品对应货位下方固定码	ArUco仍固定在货位上，但商品与货位的对应关系每局随机
任务消息	list_config、good_name、location_id、delivery_point	/supermarket_sorting/task 下发 id 和 kind
障碍物	随机投放1~2个静态障碍物	通道内随机放置5个静态障碍物
导航输入	已知地图，接口未明确	提供 odom、TF、二维激光雷达和 RGB-D
仿真运行	开发平台不限制	提供标准 Server/Client 双镜像
标准机器人	轮式双臂具身机器人	标准仿真环境采用 MMK2 轮式双臂移动机器人
示例程序	原方案未规定	提供固定 Baseline 作为参赛文件和接口示例

三、场景及商品变化

（一）货架和货位

新版场景固定设置5组货架，编号为 A、B、C、D、E。每组货架设置3层，每层设置3列：

货架：A、B、C、D、E
层号：L1、L2、L3
列号：C1、C2、C3

场景共设置：

5组货架 × 3层 × 3列 = 45个货位

货架三层高度约为0.50m、0.85m、1.19m。

（二）商品种类和数量

新版镜像设置9个商品 kind，每个 kind 5件，共45件：

kind	数量	代表物品ID
sanmingzhi	5	product_001
heweidao	5	product_002
shupian	5	product_003

kind	数量	代表物品ID
zhijin	5	product_004
maidong	5	product_005
kouxiangtang	5	product_007
pingguo	5	product_008
chengzi	5	product_009
kele	5	product_015

完整物品及模型文件位置见 `server物品清单.md`。

（三）商品位置随机化

Server 启动随机场景时，会在45个货位之间重新分配商品位置：

- 1. 商品的物理碰撞体和3DGS视觉模型同步移动；
- 2. 同一商品在不同局中可能位于不同货架、不同层和不同列；
- 3. 使用相同商品随机种子可以复现商品物理布局；
- 4. 即使商品布局相同，每次启动仍会生成新的运行前缀和匿名目标ID；
- 5. 参赛程序不得依赖固定商品坐标、固定 `product_xxx` 或上一局目标ID。

四、ArUco规则

新版镜像中，ArUco码固定绑定货位，不随商品随机化而移动或改变。

货架	ArUco ID
A	0~8
B	9~17
C	18~26
D	27~35
E	36~44

每组货架内部按 L1→L3、C1→C3 的顺序排列。例如货架D：

L1: 27、28、29
L2: 30、31、32
L3: 33、34、35

需要特别区分：

- 不随机变化的是“货位—ArUco ID”关系；
- 随机变化的是“商品—货位—ArUco ID”关系。

因此，识别到某个 ArUco ID 只能说明机器人当前观察的是哪个货位，不能直接说明该货位上是哪种商品。参赛程序仍需进行商品图像识别。

五、任务发布方式变化

（一）任务Topic

新版 Server 通过以下 ROS2 Topic 发布任务：

```
Topic: /supermarket_sorting/task
Type: std_msgs/msg/String
QoS: RELIABLE、TRANSIENT_LOCAL、depth=1
```

任务消息示例：

```
{
  "schema_version": 1,
  "run_prefix": "run_a1b2c3d4e5f6",
  "count": 5,
  "targets": [
    {"id": "item_run_a1b2c3d4e5f6_01", "kind": "kele"},
    {"id": "item_run_a1b2c3d4e5f6_02", "kind": "pingguo"},
    {"id": "item_run_a1b2c3d4e5f6_03", "kind": "chengzi"},
    {"id": "item_run_a1b2c3d4e5f6_04", "kind": "zhijin"},
    {"id": "item_run_a1b2c3d4e5f6_05", "kind": "shupian"}
  ]
}
```

字段说明：

字段	说明
schema_version	当前任务消息版本为1
run_prefix	本次 Server 进程的随机运行前缀
count	本局目标数量，正式比赛为5
targets[].id	本次 Server 进程内的匿名目标ID
targets[].kind	目标商品类别

（二）任务不包含位置信息

这是相对于原比赛方案最重要的变化。

新版任务消息不再提供：

- `location_id`；
- 货架编号；
- 层号；
- 列号；
- `aruco_id`；
- 商品世界坐标；
- “任务目标ID—货位”对应关系；
- `delivery_point` 字段。

配送目标使用场景中固定的送货区。

`targets[].id` 是匿名运行ID，不包含货架、层或列信息，不能通过解析字符串获得商品位置。该ID每次Server启动都会变化，不得跨局缓存。

参赛端能够直接用于商品视觉匹配的字段是 `kind`。参赛程序需要自主搜索货架并识别商品，建立“商品类别—货位”映射后再执行取货。

（三）建议的搜索与取货流程

参赛程序可采用以下基础流程：

接收5个任务目标



按自主规划顺序扫描货架A~E



使用ArUco确定货位



使用RGB/RGB-D识别货位上的商品kind



建立“kind-ArUco货位”映射



选择综合代价最低的目标



取一件并立即配送



继续下一件

货架不要求机械地按 A→E 顺序扫描。参赛团队可以根据起始位置、配送区位置、已识别商品、障碍绕行距离和抓取难度自主优化搜索及取送顺序。

建议保存本局已经识别的货位信息，避免每完成一次配送后重新扫描全部货架；每次实际抓取前仍应重新确认 ArUco ID 和商品类别。

六、障碍物变化

原方案规定动态障碍区随机投放1~2个静态障碍物。新版镜像调整为每局在通道内随机放置5个静态障碍物。

障碍物具有以下特点：

1. 障碍物保持箱体竖直；
2. 平面位置和偏航角随机；
3. 不与其他障碍物重叠；
4. 与墙体保持安全距离；
5. 场景生成器检查货架入口至配送台入口存在可通行路线；
6. 障碍布局会形成需要实际绕行的路径。

固定 Baseline 使用固定航点，不处理随机障碍物。参赛程序需要使用二维激光雷达、里程计和 TF 实现自主规划、避障和重规划。

七、Server/Client双镜像

（一）Server镜像

```
crpi-1pzq998p9m7w0auy.cn-hangzhou.personal.cr.aliyuncs.com/  
challengecup/supermarket_sorting_final:server  
  
sha256:eb0b58a600b85910c2e5392852e9268ca6a5aaca02385ab8a0f69981d833c4b2
```

Server负责：

- 加载 MuJoCo 场景；
- 生成商品和障碍物布局；
- 发布任务及传感器数据；
- 接收机器人控制命令。

（二）Client镜像

```
crpi-1pzq998p9m7w0auy.cn-hangzhou.personal.cr.aliyuncs.com/  
challengecup/supermarket_sorting_final:client
```

sha256:db0bfd2c75e34af430e2e79f07c0ab9528d843e6fc6b8ce067216fd28168f0e

Client提供统一运行环境：

组件	版本
操作系统	Ubuntu 22.04
CUDA	12.8
Python	3.10
PyTorch	2.7.1+cu128
torchvision	0.22.1+cu128
ROS2	Humble
RMW	CycloneDDS
NumPy	1.26.4
SciPy	1.11.4
OpenCV	4.10.0
Ultralytics	8.0.196

Client 中的 `/workspace/baseline` 是参赛代码挂载目录。Client 镜像不内置参赛源码属于正常设计，各团队应挂载固定 Baseline 或本队程序。

Server 和 Client 必须使用：

```
ROS_DOMAIN_ID=99
RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

镜像加载和启动命令见 `智慧零售_README.md`。

八、ROS2接口变化

（一）Server发布

Topic	Type	说明
<code>/supermarket_sorting/task</code>	<code>std_msgs/msg/String</code>	JSON任务清单
<code>/slamware_ros_sdk_server_node/odom</code>	<code>nav_msgs/msg/Odometry</code>	底盘位姿和速度
<code>/tf</code>	<code>tf2_msgs/msg/TFMessage</code>	动态TF
<code>/slamware_ros_sdk_server_node/scan</code>	<code>sensor_msgs/msg/LaserScan</code>	360点二维激光雷达，默认12Hz

Topic	Type	说明
/joint_states	sensor_msgs/msg/JointState	关节状态
/head_camera/color/image_raw	sensor_msgs/msg/Image	头部RGB图像
/head_camera/color/camera_info	sensor_msgs/msg/CameraInfo	头部相机内参
/head_camera/aligned_depth_to_color/image_raw	sensor_msgs/msg/Image	对齐深度图，单位毫米
/head_camera/aligned_depth_to_color/camera_info	sensor_msgs/msg/CameraInfo	深度相机内参
/left_camera/color/image_raw	sensor_msgs/msg/Image	左腕RGB图像
/left_camera/color/camera_info	sensor_msgs/msg/CameraInfo	左腕相机内参
/right_camera/color/image_raw	sensor_msgs/msg/Image	右腕RGB图像
/right_camera/color/camera_info	sensor_msgs/msg/CameraInfo	右腕相机内参

二维激光雷达当前参数：

扫描点数：360
发布频率：12Hz
frame: laser
最小有效距离：0.02m
最大有效距离：12m

(二) Server订阅

Topic	Type	控制格式
/cmd_vel	geometry_msgs/msg/Twist	linear.x、angular.z
/spine_forward_position_controller/commands	std_msgs/msg/Float64MultiArray	[slide_joint]
/head_forward_position_controller/commands	std_msgs/msg/Float64MultiArray	[head_yaw_joint, head_pitch_joint]
/left_arm_forward_position_controller/commands	std_msgs/msg/Float64MultiArray	[joint1..joint6, gripper]
/right_arm_forward_position_controller/commands	std_msgs/msg/Float64MultiArray	[joint1..joint6, gripper]

九、Baseline示例定位

固定 Baseline 用于展示：

1. 参赛文件目录结构；
2. Client代码挂载方式；
3. ROS2节点启动方式；
4. RGB-D目标检测；
5. 目标世界坐标计算；
6. MMK2关节状态读取；
7. 底盘、升降柱、机械臂和夹爪控制；
8. 一次固定目标抓取流程。

固定 Baseline 使用固定目标、固定货位和固定航点，不处理随机商品、随机障碍或完整5订单任务。这是示例范围，不表示相关赛题要求被取消，也不限制参赛团队采用其他算法或程序结构。

固定示例模式典型参数：

```
SUPERMARKET_FIXED_BASELINE=1
SUPERMARKET_RANDOMIZE=0
SUPERMARKET_RANDOMIZE_OBSTACLES=0
SUPERMARKET_TASKS=product_032
```

完整随机场景测试参数：

```
SUPERMARKET_RANDOMIZE=1
SUPERMARKET_RANDOMIZE_OBSTACLES=1
SUPERMARKET_TASKS=all
```

以上完整场景参数用于本地开发测试，不代表正式赛时固定 seed 或目标数量。

十、参赛程序适配要求

各参赛团队应根据上述变化完成以下适配：

1. 在官方 Client 镜像中运行和验证参赛程序；
2. 将参赛代码挂载到 `/workspace/baseline`；
3. 订阅并解析 `/supermarket_sorting/task`；
4. 不再依赖原方案中的 `location_id`；
5. 不硬编码商品位置、ArUco对应商品或运行目标ID；
6. 自主扫描货架并建立“商品类别—货位”映射；

7. 使用 ArUco 确定货位，使用视觉识别商品；
8. 使用 LaserScan、odom 和 TF 进行自主规划及避障；
9. 保持“取一件—送一件—再取下一件”的任务闭环；
10. 在多个商品随机种子和障碍随机种子下测试；
11. 对无检测、深度无效、规划失败、抓取失败和通信中断进行安全处理；
12. 不读取 Server 内部布局文件或其他未公开真值。

本说明未明确变更的内容，继续按原比赛方案执行。